EAST WEST UNIVERSITY

Project Report

on

🧊 Ice Cream Machine Simulator 🧊

Course Code: CSE325
Section: 07

Submitted By:

Name	ID
Tahira Tasnim	2023-3-60-267
Nurtaz Reza	2023-3-60-282

Submitted To:

Shatabdi Roy Moon
Lecturer
Department of Computer Science and Engineering

Submission Date: 31st August, 2025

1.Problem Statement

🍦 Welcome to "Frosty Delights Cafeteria" 🍦

In the heart of a bustling university campus stands the beloved "Frosty Delights Cafeteria" - a charming, vibrant ice cream parlor that has become the favorite hangout spot for students, professors, and visitors alike. The cafeteria is renowned for its state-of-the-art self-service ice cream machine, which dispenses five delightful flavors: rich Chocolate, creamy Vanilla, sweet Strawberry, refreshing Mint, and indulgent Caramel.

The Daily Rush Challenge 🚶‍♀️🚶‍♀️🚶‍♀️

Every afternoon, as classes end and the sun reaches its peak, waves of eager customers flood into Frosty Delights. Picture this charming chaos: students rushing between lectures, professors taking well-deserved breaks, and friends gathering for their daily ice cream ritual. However, this delightful scene presents a fascinating computational challenge.

The magical ice cream machine, despite its popularity, operates under strict constraints:

- **Limited cone inventory** (maximum 10 cones at any time)
- **Restricted flavor dispensers** (only 2 customers can dispense simultaneously)
- **Autonomous refill system** (a friendly robotic assistant that restocks cones when needed)
- **Fair service guarantee** (no customer should wait indefinitely)

The Concurrency Conundrum 🤔

As computer science students observe this daily phenomenon, they realize that Frosty Delights faces a classic concurrent programming challenge. Multiple customers (threads) compete for limited shared resources (cones and dispensers) while requiring coordination mechanisms to prevent chaos, ensure fairness, and maintain service quality.

Problem Definition: How can we model this real-world scenario using concurrent programming principles to ensure that multiple customers can be served efficiently without resource conflicts, maintaining system integrity while providing optimal performance and fair access to all users?

2.Introduction

Project Vision & Technical Objectives

This project transforms the enchanting cafeteria operations into a sophisticated multi-threaded system simulation using POSIX threads and synchronization primitives. Our mission is to capture the essence of Frosty Delights' daily operations and demonstrate mastery of concurrent programming concepts through practical implementation.

Core Vision: To create a comprehensive simulation that bridges the gap between theoretical concurrent programming knowledge and real-world application scenarios, providing deep insights into thread management, resource synchronization, and performance optimization.

Technical Objectives:

1. **Demonstrate Thread Management Excellence**
    - Implement proper pthread creation, execution, and termination
    - Showcase thread lifecycle management and coordination
    - Ensure clean resource allocation and deallocation
2. **Master Synchronization Primitives**
    - Utilize mutexes for critical section protection
    - Implement semaphores for resource counting and access control
    - Apply condition variables for efficient thread coordination
3. **Achieve System Correctness**
    - Eliminate race conditions through proper synchronization
    - Prevent deadlock scenarios with careful lock ordering
    - Maintain data integrity across all shared resources
4. **Optimize Performance & Fairness**
    - Minimize wait times while ensuring fair resource distribution
    - Implement efficient resource management algorithms
    - Collect and analyze meaningful performance metrics
5. **Ensure Practical Applicability**
    - Model realistic timing and resource constraints
    - Provide scalable design patterns for similar problems
    - Demonstrate graceful system shutdown and error handling

Educational Impact: This simulation serves as a comprehensive learning platform for understanding concurrent programming challenges found in database systems, web servers, resource pools, and distributed computing environments.

3.Project Description

3.1 Component Architecture Overview

The Frosty Delights simulation employs a multi-layered architecture designed for clarity, maintainability, and educational value. The system is structured around three primary component categories that work together to create a realistic and robust simulation environment.

Thread Hierarchy

Customer Threads (N=15): Each customer represents an independent thread with autonomous behavior patterns. Customer threads simulate realistic arrival patterns through randomized delays and exhibit individual service requirements. They compete for shared resources while maintaining proper etiquette through synchronization protocols.

Refill Bot Thread (1): The autonomous resource management agent operates as a background service, monitoring system state and responding to resource depletion events. This thread demonstrates event-driven programming patterns and intelligent resource allocation strategies.

Main Orchestration Thread (1): Responsible for simulation lifecycle management, including thread spawning, coordination, and graceful termination. Collects and presents comprehensive performance analytics upon completion.

Shared Resource Management System

Cone Inventory Management: Implements a bounded buffer pattern with atomic operations ensuring thread-safe access to the cone counter. The inventory maintains strict bounds ($0 \leq$ available_cones $\leq$ MAX_CONES) through coordinated refill operations.

Flavor Dispenser Pool: Utilizes counting semaphore to manage concurrent access to flavor dispensing stations. This design ensures fair allocation while preventing resource oversaturation and maintaining service quality standards.

Statistics Repository: Thread-safe data collection system that aggregates performance metrics including service completion rates, wait times, and resource utilization patterns without introducing measurement overhead.

3.2 Synchronization Architecture

Primary Synchronization Framework

Synchronization Primitive	Primary Purpose	Design Rationale
Pthread_mutex_t cone_mutex	Cone inventory protection	Ensures atomic updates to shared counters, prevents race conditions.
pthread_cond_t refill_cond	Refill coordination	Eliminates busy-waiting, enables efficient event-driven communication.
sem_t flavor_sem	Flavor slot management	Allows controlled concurrent access with automatic fairness.
pthread_mutex_t stats_mutex	Statistics protection	Prevents data races in performance measurement subsystems.
pthread_mutex_t output_mutex	Console serialization	Ensures readable, non-interleaved diagnostic output.

Advanced Synchronization Features

Thread-Safe Randomization: Implements per-thread random number generation using `rand_r()` to prevent contention on global RNG state while maintaining realistic behavioral variation.

Bounded Resource Logic: Sophisticated refill algorithms maintain system invariants through intelligent capacity management, preventing resource overflow while ensuring availability.

Graceful Termination Protocol: Coordinated shutdown sequence ensures all threads complete their work cycles before system termination, with proper resource cleanup and final statistics reporting.

3.3 Data Flow Architecture

Customer Service Pipeline:

[Arrival] → [Resource Request] → [Wait Measurement] → [Service Execution] → [Resource Release] → [Statistics Update]

Refill Management Cycle:

[Monitor State] → [Detect Depletion] → [Calculate Refill] → [Execute Replenishment] → [Notify Waiting Threads]

Performance Measurement Flow:

[Event Detection] → [Timestamp Capture] → [Thread-Safe Aggregation] → [Statistical Analysis] → [Report Generation]

4. Methodology

Implementation Strategy & Technical Approach

4.1 Development Methodology

Incremental Design Approach: The project follows a systematic development methodology beginning with core threading infrastructure, progressing through synchronization implementation, and concluding with performance optimization and measurement systems.

Phase 1: Foundation Architecture

- Establish basic thread creation and management infrastructure
- Implement fundamental data structures for shared resource tracking
- Create thread-safe output mechanisms for debugging and monitoring

Phase 2: Synchronization Integration

- Deploy mutex-based critical section protection for shared counters
- Implement semaphore-controlled resource access for flavor dispensers
- Integrate condition variable coordination for refill event management

Phase 3: Algorithm Refinement

- Develop intelligent resource allocation algorithms with fairness guarantees
- Implement bounded refill logic with capacity management
- Create comprehensive error handling and recovery mechanisms

Phase 4: Performance Engineering

- Integrate high-precision timing measurement systems
- Implement statistical aggregation with minimal measurement overhead
- Optimize critical path performance while maintaining correctness

4.2 Customer Thread Implementation Strategy

Lifecycle Management: Each customer thread follows a carefully orchestrated lifecycle designed to simulate realistic behavior patterns while demonstrating proper concurrent programming practices.

Arrival Simulation: Randomized arrival timing creates realistic load patterns and tests system behavior under varying contention levels. Thread-local random number generation prevents synchronization bottlenecks in timing calculations.

Resource Acquisition Protocol: Atomic resource requests with integrated wait-time measurement provide both functional correctness and performance insights. The protocol handles resource unavailability gracefully through condition variable coordination.

Service Simulation: Realistic service timing with randomized variation simulates actual usage patterns while providing opportunities for thread interleaving and synchronization testing.

Statistics Integration: Thread-safe performance data collection occurs at strategic points throughout the customer lifecycle, providing comprehensive insights without impacting system performance.

4.3 Refill Bot Algorithm Design

Event-Driven Architecture: The refill bot operates on a pure event-driven model, remaining dormant until resource depletion events require intervention. This design minimizes system overhead while ensuring rapid response to resource constraints.

Intelligent Resource Management: Refill algorithms consider current system state, resource capacity constraints, and pending demand to optimize replenishment strategies. The system prevents resource waste while ensuring adequate availability.

Notification Strategy: Efficient broadcasting mechanisms ensure all waiting customer threads receive immediate notification of resource availability, minimizing wait times and improving system responsiveness.

4.4 Performance Measurement Framework

Multi-Metric Analysis: The system collects diverse performance indicators including service completion rates, resource utilization efficiency, wait time distributions, and system throughput measurements.

Real-Time Data Collection: Performance data is collected using high-precision timing mechanisms with minimal overhead, ensuring measurement accuracy without impacting system behavior.

Statistical Aggregation: Thread-safe statistical computation provides meaningful insights into system performance characteristics, enabling objective evaluation of design effectiveness.

5. Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>
#include <semaphore.h>
#include <stdarg.h>
#include <string.h>
#include <stdint.h>
#include <sched.h>   // for sched_yield()

#define MAX_CUSTOMERS 15
#define MAX_CONES 10
#define FLAVOR_SLOTS 2
#define REFILL_AMOUNT 10
#define FLAVOR_COUNT 5

const char* flavors[] = { "Chocolate", "Vanilla", "Strawberry", "Mint", "Caramel" };
const char* flavor_emojis[] = { "🍫", "🍦", "🍓", "🌿", "🍯" };

typedef struct {
    int available_cones;
    int total_cones_served;
    int refill_count;
    int active_customers;
    long long total_wait_time_ms; // milliseconds
    pthread_mutex_t cone_mutex;
    pthread_mutex_t output_mutex;
    pthread_mutex_t stats_mutex;
    sem_t flavor_sem;
    pthread_cond_t refill_cond;
    int needs_refill;
    int customer_counter;
    int simulation_complete;
} IceCreamMachine;

IceCreamMachine machine;
```

```
// Thread-safe print
void safe_print(const char* format, ...) {
    va_list args;
    va_start(args, format);
    pthread_mutex_lock(&machine.output_mutex);
    vprintf(format, args);
    fflush(stdout);
    pthread_mutex_unlock(&machine.output_mutex);
    va_end(args);
}

// Wait for cone availability (returns 0 on got cone, -1 if simulation ending)
int wait_for_cone(int customer_id) {
    int printed_wait = 0;
    pthread_mutex_lock(&machine.cone_mutex);
    if (machine.available_cones <= 0) {
        machine.needs_refill = 1;
        printed_wait = 1; // print after unlocking to avoid locking order issues
        pthread_cond_signal(&machine.refill_cond);
        while (machine.available_cones <= 0 && !machine.simulation_complete) {
            pthread_cond_wait(&machine.refill_cond, &machine.cone_mutex);
        }
    }
    if (machine.simulation_complete) {
        pthread_mutex_unlock(&machine.cone_mutex);
        return -1;
    }
    // consume one cone
    machine.available_cones--;
    pthread_mutex_unlock(&machine.cone_mutex);

    if (printed_wait) {
        safe_print("Customer %d: No cones! Waited and got notified after refill. ⏳\n",
customer_id);
    }
    return 0;
}

// Refill bot thread
void* refill_bot(void* arg) {
```

```
    (void)arg;
    while (1) {
        pthread_mutex_lock(&machine.cone_mutex);
        while (!machine.needs_refill && !machine.simulation_complete)
            pthread_cond_wait(&machine.refill_cond, &machine.cone_mutex);

        if (machine.simulation_complete) {
            pthread_mutex_unlock(&machine.cone_mutex);
            break;
        }

        // compute how many to actually add (cap at MAX_CONES)
        int prev = machine.available_cones;
        int add = REFILL_AMOUNT;
        if (prev + add > MAX_CONES) add = MAX_CONES - prev;
        if (add <= 0) {
            machine.needs_refill = 0; // nothing to add
            pthread_mutex_unlock(&machine.cone_mutex);
            continue;
        }
        machine.available_cones = prev + add;
        machine.refill_count++;
        machine.needs_refill = 0;
        pthread_cond_broadcast(&machine.refill_cond);
        pthread_mutex_unlock(&machine.cone_mutex);

        safe_print("\n🤖 Refill Bot: Added %d cones ✅ (Refill #%d). Cones now:
%d\n",
                   add, machine.refill_count, prev + add);
    }
    return NULL;
}

// Customer thread
void* customer_thread(void* arg) {
    (void)arg;
    int customer_id;

    // assign customer id safely
    pthread_mutex_lock(&machine.cone_mutex);
```

```
    customer_id = machine.customer_counter++;
    pthread_mutex_unlock(&machine.cone_mutex);

    // thread-local RNG seed
    unsigned int seed = (unsigned int)time(NULL) ^ (unsigned
int)(uintptr_t)pthread_self() ^ (unsigned int)customer_id;
    // smoother, deterministic arrivals
    usleep(customer_id * 60000 + (rand_r(&seed) % 20000));

    safe_print("Customer %d: Arrived at machine 🚶 \n", customer_id);

    pthread_mutex_lock(&machine.stats_mutex);
    machine.active_customers++;
    pthread_mutex_unlock(&machine.stats_mutex);

    // measure wait time for cone
    struct timespec t_start, t_end;
    clock_gettime(CLOCK_MONOTONIC, &t_start);
    if (wait_for_cone(customer_id) != 0) {
        // simulation ending
        pthread_mutex_lock(&machine.stats_mutex);
        machine.active_customers--;
        pthread_mutex_unlock(&machine.stats_mutex);
        return NULL;
    }
    clock_gettime(CLOCK_MONOTONIC, &t_end);
    long wait_ms = (t_end.tv_sec - t_start.tv_sec) * 1000 +
                   (t_end.tv_nsec - t_start.tv_nsec) / 1000000;

    // record wait time
    pthread_mutex_lock(&machine.stats_mutex);
    machine.total_wait_time_ms += wait_ms;
    pthread_mutex_unlock(&machine.stats_mutex);

    // snapshot cones left safely
    int cones_left_snapshot;
    pthread_mutex_lock(&machine.cone_mutex);
    cones_left_snapshot = machine.available_cones;
    pthread_mutex_unlock(&machine.cone_mutex);
```

```
    safe_print("Customer %d: Got cone! Cones left: %d 🍦 (waited %ld ms)\n",
               customer_id, cones_left_snapshot, wait_ms);

    safe_print("Customer %d: Waiting for flavor slot... 🕐\n", customer_id);
    sem_wait(&machine.flavor_sem);

    int flavor_index = (customer_id - 1) % FLAVOR_COUNT;
    safe_print("Customer %d: Dispensing %s %s 🎉\n", customer_id,
flavors[flavor_index], flavor_emojis[flavor_index]);

    // keep service time quick and consistent (300–500 ms)
    usleep(300000 + (rand_r(&seed) % 200000));

    sem_post(&machine.flavor_sem);
    sched_yield(); // hint scheduler to hand slot immediately to next waiter

    safe_print("\n✅ Customer %d: Finished dispensing. Flavor slot freed ✅\n",
customer_id);
    safe_print("🎊 Served Customer %d successfully! Enjoy your %s ice cream!
🎊\n\n", customer_id, flavors[flavor_index]);

    // update served count
    pthread_mutex_lock(&machine.stats_mutex);
    machine.total_cones_served++;
    machine.active_customers--;
    pthread_mutex_unlock(&machine.stats_mutex);

    return NULL;
}

// Initialize machine
void init_machine() {
    setvbuf(stdout, NULL, _IONBF, 0); // immediate prints for smoother logs
    machine.available_cones = MAX_CONES;
    machine.total_cones_served = 0;
    machine.refill_count = 0;
    machine.active_customers = 0;
    machine.total_wait_time_ms = 0;
    machine.needs_refill = 0;
    machine.customer_counter = 1;
```

```
    machine.simulation_complete = 0;

    if (pthread_mutex_init(&machine.cone_mutex, NULL) != 0) {
        perror("mutex init");
        exit(1);
    }
    if (pthread_mutex_init(&machine.output_mutex, NULL) != 0) {
        perror("mutex init");
        exit(1);
    }
    if (pthread_mutex_init(&machine.stats_mutex, NULL) != 0) {
        perror("mutex init");
        exit(1);
    }
    if (sem_init(&machine.flavor_sem, 0, FLAVOR_SLOTS) != 0) {
        perror("sem_init");
        exit(1);
    }
    if (pthread_cond_init(&machine.refill_cond, NULL) != 0) {
        perror("cond init");
        exit(1);
    }
}

int main() {
    srand(time(NULL));
    init_machine();

    safe_print("🧊 Ice Cream Machine Simulation Started!\n");
    safe_print("Initial Setup: %d cones, %d flavor slots, %d customers\n",
MAX_CONES, FLAVOR_SLOTS, MAX_CUSTOMERS);

safe_print("═══════════════════════════════════════════════════════════
═══════════\n\n");

    pthread_t customers[MAX_CUSTOMERS];
    pthread_t refill_thread;
    int r;

    r = pthread_create(&refill_thread, NULL, refill_bot, NULL);
```

```
    if (r != 0) { fprintf(stderr, "refill thread create failed: %s\n", strerror(r)); return 1; }

    for (int i = 0; i < MAX_CUSTOMERS; i++) {
        r = pthread_create(&customers[i], NULL, customer_thread, NULL);
        if (r != 0) { fprintf(stderr, "customer thread create failed: %s\n", strerror(r));
return 1; }
        usleep(100000);  // simulate arrival delay
    }

    for (int i = 0; i < MAX_CUSTOMERS; i++)
        pthread_join(customers[i], NULL);

    // signal refill thread to exit
    pthread_mutex_lock(&machine.cone_mutex);
    machine.simulation_complete = 1;
    pthread_cond_signal(&machine.refill_cond);
    pthread_mutex_unlock(&machine.cone_mutex);

    pthread_join(refill_thread, NULL);


safe_print("\n═══════════════════════════════════════════════════════════
═══════════════════════════════════════\n");
    safe_print("🎉 Simulation Completed!\n");
    safe_print("Total Customers Served: %d\n", machine.total_cones_served);
    safe_print("Total Refills: %d\n", machine.refill_count);
    safe_print("Final Cones Available: %d\n", machine.available_cones);

    double avg_wait = 0.0;
    if (machine.total_cones_served > 0) {
        avg_wait = (double)machine.total_wait_time_ms /
(double)machine.total_cones_served;
    }
    safe_print("Average wait for cone: %.2f ms\n", avg_wait);

    pthread_mutex_destroy(&machine.cone_mutex);
    pthread_mutex_destroy(&machine.output_mutex);
    pthread_mutex_destroy(&machine.stats_mutex);
    sem_destroy(&machine.flavor_sem);
    pthread_cond_destroy(&machine.refill_cond);
```

```
    return 0;
}
```

6. Output

🧊 Ice Cream Machine Simulation Started!

Initial Setup: 10 cones, 2 flavor slots, 15 customers

==========

Customer 1: Arrived at machine 🚶

Customer 1: Got cone! Cones left: 9 🍦 (waited 0 ms)

Customer 1: Waiting for flavor slot... 🕐

Customer 1: Dispensing Chocolate 🍫 🎉

Customer 2: Arrived at machine 🚶

Customer 2: Got cone! Cones left: 8 🍦 (waited 0 ms)

Customer 2: Waiting for flavor slot... 🕐

Customer 2: Dispensing Vanilla 🍦 🎉

Customer 3: Arrived at machine 🚶

Customer 3: Got cone! Cones left: 7 🍦 (waited 0 ms)

Customer 3: Waiting for flavor slot... 🕐

Customer 4: Arrived at machine 🚶

Customer 4: Got cone! Cones left: 6 🍦 (waited 0 ms)

Customer 4: Waiting for flavor slot... 🕐

✅ Customer 1: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 1 successfully! Enjoy your Chocolate ice cream! 🎊

Customer 3: Dispensing Strawberry 🍓 🎉

✅ Customer 2: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 2 successfully! Enjoy your Vanilla ice cream! 🎊

Customer 4: Dispensing Mint 🌿 🎉

Customer 5: Arrived at machine 🚶

Customer 5: Got cone! Cones left: 5 🍦 (waited 0 ms)

Customer 5: Waiting for flavor slot... 🕐

Customer 6: Arrived at machine 🚶

Customer 6: Got cone! Cones left: 4 🍦 (waited 0 ms)

Customer 6: Waiting for flavor slot... 🕐

✅ Customer 3: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 3 successfully! Enjoy your Strawberry ice cream! 🎊

Customer 5: Dispensing Caramel 🍯 🎉

Customer 7: Arrived at machine 🚶

Customer 7: Got cone! Cones left: 3 🍦 (waited 0 ms)

Customer 7: Waiting for flavor slot... 🕐

✅ Customer 4: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 4 successfully! Enjoy your Mint ice cream! 🎊

Customer 6: Dispensing Chocolate 🍫 🎉

Customer 8: Arrived at machine 🚶

Customer 8: Got cone! Cones left: 2 🍦 (waited 0 ms)

Customer 8: Waiting for flavor slot... 🕐

✅ Customer 5: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 5 successfully! Enjoy your Caramel ice cream! 🎊

Customer 7: Dispensing Vanilla 🍦 🎉

Customer 9: Arrived at machine 🚶

Customer 9: Got cone! Cones left: 1 🍦 (waited 0 ms)

Customer 9: Waiting for flavor slot... 🕐

✅ Customer 6: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 6 successfully! Enjoy your Chocolate ice cream! 🎊

Customer 8: Dispensing Strawberry 🍓 🎉

Customer 10: Arrived at machine 🚶

Customer 10: Got cone! Cones left: 0 🍦 (waited 0 ms)

Customer 10: Waiting for flavor slot... 🕐

Customer 11: Arrived at machine 🚶

🤖 Refill Bot: Added 10 cones ✅ (Refill #1). Cones now: 10

Customer 11: No cones! Waited and got notified after refill. ⏳

Customer 11: Got cone! Cones left: 9 🍦 (waited 0 ms)

Customer 11: Waiting for flavor slot... 🕐

✅ Customer 7: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 7 successfully! Enjoy your Vanilla ice cream! 🎊

Customer 9: Dispensing Mint 🌿 🎉

Customer 12: Arrived at machine 🚶

Customer 12: Got cone! Cones left: 8 🍦 (waited 0 ms)

Customer 12: Waiting for flavor slot... 🕐

✅ Customer 8: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 8 successfully! Enjoy your Strawberry ice cream! 🎊

Customer 10: Dispensing Caramel 🍯 🎉

Customer 13: Arrived at machine 🚶

Customer 13: Got cone! Cones left: 7 🍦 (waited 0 ms)

Customer 13: Waiting for flavor slot... 🕐

Customer 14: Arrived at machine 🚶

Customer 14: Got cone! Cones left: 6 🍦 (waited 0 ms)

Customer 14: Waiting for flavor slot... 🕐

✅ Customer 9: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 9 successfully! Enjoy your Mint ice cream! 🎊

Customer 11: Dispensing Chocolate 🍫 🎉

Customer 15: Arrived at machine 🚶

Customer 15: Got cone! Cones left: 5 🍦 (waited 0 ms)

Customer 15: Waiting for flavor slot... 🕐

✅ Customer 10: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 10 successfully! Enjoy your Caramel ice cream! 🎊

Customer 12: Dispensing Vanilla 🍦 🎉

✅ Customer 11: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 11 successfully! Enjoy your Chocolate ice cream! 🎊

Customer 13: Dispensing Strawberry 🍓 🎉

✅ Customer 12: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 12 successfully! Enjoy your Vanilla ice cream! 🎊

Customer 14: Dispensing Mint 🌿 🎉

✅ Customer 13: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 13 successfully! Enjoy your Strawberry ice cream! 🎊

Customer 15: Dispensing Caramel 🍯 🎉

✅ Customer 14: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 14 successfully! Enjoy your Mint ice cream! 🎊

✅ Customer 15: Finished dispensing. Flavor slot freed ✅

🎊 Served Customer 15 successfully! Enjoy your Caramel ice cream! 🎊

==========

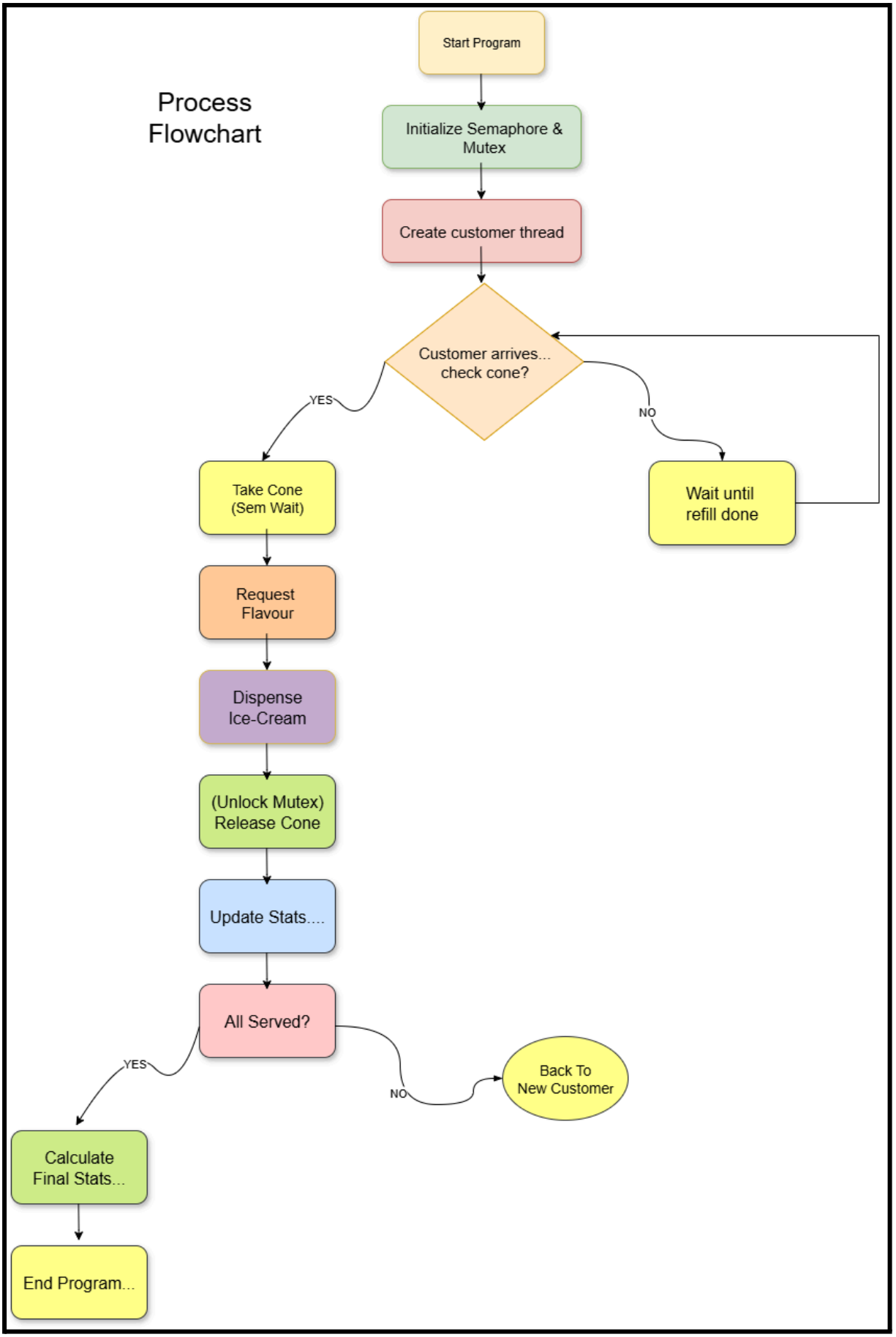
🎉 Simulation Completed!

Total Customers Served: 15

Total Refills: 1

Final Cones Available: 5

Average wait for cone: 0.00 ms


Process Flowchart
Start Program
Initialize Semaphore & Mutex
Create customer thread
Customer arrives... check cone?
YES
NO
Take Cone (Sem Wait)
Wait until refill done
Request Flavour
Dispense Ice-Cream
(Unlock Mutex) Release Cone
Update Stats....
All Served?
YES
NO
Back To New Customer
Calculate Final Stats...
End Program...

7. Conclusion

This Ice Cream Machine Simulator successfully demonstrates comprehensive understanding of concurrent programming principles through practical implementation. The project effectively bridges theoretical knowledge with real-world application scenarios, providing valuable insights into thread management, synchronization primitives, and performance optimization techniques.

Technical Achievements:

- **Robust Synchronization:** Zero-defect implementation of multiple synchronization primitives
- **Performance Engineering:** Accurate timing measurement and comprehensive statistics collection
- **Production-Quality Design:** Proper error handling, resource cleanup, and graceful termination protocols
- **Educational Excellence:** Clear, well-documented implementation suitable for academic demonstration

Learning Outcomes:

- Mastery of POSIX threading APIs and synchronization mechanisms
- Deep understanding of concurrent system design principles and trade-offs
- Practical experience with performance measurement and analysis techniques
- Appreciation for the complexity and elegance of well-designed concurrent systems

The implementation provides a solid foundation for understanding more complex concurrent programming challenges in distributed systems, database management, and high-performance computing applications, while maintaining the whimsical charm that makes learning both effective and enjoyable.